

Lab 06: Cloud Scheduling with Reinforcement Learning

Mon hoc: Hoc May Tang Cuong cho cac He Thong Mang
GVHD: ThS. Phan Trung Phat

Cau 5: Yeu to MDP, Luong Hoat Dong va Cach Van Hanh

5.1 Cac thanh phan MDP

Bai toan cloud scheduling duoc mo hinh hoa thanh Markov Decision Process (MDP) voi cac thanh phan sau:

State (Trang thai)

Vector quan sat co 3 nhom dac trung. Voi 200 servers x 5 VMs:

$$\text{state_size} = 3 + 6 + (200 \times 5 \times 3) = 3009$$

Nhom	So chieu	Mo ta
Summary features	3	$\text{current_time}/\text{max_steps}$, $\text{ready_queue}/\text{total_tasks}$, $\text{running_tasks}/\text{total_tasks}$
Task features	6	cpu_demand , ram_demand , disk_demand , normalized runtime, normalized deadline, $\text{wait_time}/\text{max_steps}$
Resource features	3000	CPU/RAM/Disk con lai cua tung VM tren moi server

Action (Hanh dong)

- $[0]$ den $[999]$: chon cap $(\text{server_index}, \text{vm_index})$ de gan task (200 servers x 5 VMs = 1000 slots)
- $[1000]$ (wait_action): bo qua 1 buoc thoi gian, khong len lich task nao

Reward (Phan thuong)

Tinh huong	Reward	Ly do
Schedule thanh cong	$+1.0 + \text{urgency_bonus}$	Khuyen khich len lich som, giam completion time
Wait	-0.05	Phat nhe de giam waiting time
Reject	-1.0	Phat nang de giam rejection rate

Transition & Terminal

- Moi `step()`: release completed tasks $\rightarrow$ xu ly action $\rightarrow$ `advance_time(+1.0)`
- Episode ket thuc khi: `ready_queue` rong + khong con running task + toan bo tasks da handled

5.2 Luong hoat dong

```

reset()
├─ load job_templates tu dataset.txt
├─ khoi tao resources[200][5][3] = 0.2 (moi VM = 1/5 capacity)
└─ dua tat ca tasks vao ready_queue

step(action):
├─ _release_completed_tasks()      # giai phong tai nguyen
├─ _finalize_ready_task(action):
│   ├─ wait_action                → reward = -0.05
│   ├─ valid + du tai nguyen → _allocate_task() → reward > 0
│   └─ invalid / thieu tai nguyen → _reject_task() → reward = -1.0
├─ _advance_time()                # current_time += 1.0
└─ check terminated / truncated

```

Cau 1: Dinh nghia Reward Function

Reward duoc thiet ke de dong thoi toi thieu hoa completion time, waiting time va rejection rate:

```
python
```

```
def _calculate_reward(self, task, success, reason=None):
    if success:
        # Thuong co ban cho viec schedule thanh cong
        base_reward = 1.0

        # Urgency bonus: thuong them neu con nhieu thoi gian truoc deadline
        # Khuyen khich agent uu tien schedule som de giam completion time
        time_margin = task.deadline - (self.current_time + task.runtime)
        urgency_bonus = 0.5 * max(0.0, time_margin / task.deadline) if task.deadline

        # Waiting penalty: phat nhe theo thoi gian cho trong queue
        wait_time = self._task_wait_time(task)
        wait_penalty = -0.01 * (wait_time / float(max(1, self.max_steps)))

        return base_reward + urgency_bonus + wait_penalty
    else:
        if reason == "wait":
            # Phat nhe khi cho -> tranh tinh trang agent toan "wait"
            return -0.05
        else:
            # Phat nang khi reject -> giam rejection rate
            return -1.0
```

Cau 6: Thuat Toan Duoc Chon va Ly Do

Hai thuat toan: DQN va PPO

Tieu chi	DQN	PPO
Loai action space	Discrete (1001 actions)	Discrete (1001 actions)
State space lon (3009 chieu)	Neural network function approximator	Neural network function approximator
Sample efficiency	Cao — replay buffer tai su dung experience	Trung binh — on-policy, moi rollout dung vai epochs
Stability	Target network tranh oscillation	Clip ratio tranh update qua lon
Valid action masking	Co, trong epsilon-greedy	Co, mask logits truoc khi sample
Hoi tu	Cham hon nhung on dinh	Nhanh hon trong episode dau

Ly do chon DQN lam thuat toan chinh:

- Action space roi rac lon (1001 actions) — DQN xu ly tot voi `argmax` tren Q-values
- Replay buffer giup tai su dung experience hieu qua, phu hop khi so episode han che
- Target network giu on dinh gradient, tranh diverge khi reward co the sparse
- De ket hop valid action masking: chi can mask cac action khong hop le khi chon `argmax`

PPO duoc dung de so sanh vi hoi tu nhanh hon o cac episode dau va entropy bonus tu dong khuyen khich exploration.

Cau 2 & 3: Code Day Du

Import va Setup

```
python

import copy
import os
import random
from collections import deque

import gymnasium as gym
import numpy as np
from gymnasium import spaces
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
```

Reward Function (dien vao CloudSchedulingEnv)

```
python
```

```
def _calculate_reward(self, task, success, reason=None):
    if success:
        base_reward = 1.0
        time_margin = task.deadline - (self.current_time + task.runtime)
        urgency_bonus = 0.5 * max(0.0, time_margin / task.deadline) if task.deadline
        wait_time = self._task_wait_time(task)
        wait_penalty = -0.01 * (wait_time / float(max(1, self.max_steps)))
        return base_reward + urgency_bonus + wait_penalty
    else:
        return -0.05 if reason == "wait" else -1.0
```

DQN Agent

python

```

class DQNNetwork(nn.Module):
    """Neural network cho DQN - 2 hidden layers."""
    def __init__(self, state_size, action_size, hidden=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_size, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, action_size)
        )

    def forward(self, x):
        return self.net(x)

class ReplayBuffer:
    """Experience replay buffer."""
    def __init__(self, capacity=50_000):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        batch = random.sample(self.buffer, batch_size)
        s, a, r, ns, d = zip(*batch)
        return (np.array(s, dtype=np.float32),
                np.array(a),
                np.array(r, dtype=np.float32),
                np.array(ns, dtype=np.float32),
                np.array(d, dtype=np.float32))

    def __len__(self):
        return len(self.buffer)

class DQNAgent:
    def __init__(self, state_size, action_size, lr=1e-3, gamma=0.99,
                 epsilon=1.0, epsilon_min=0.05, epsilon_decay=0.995,
                 batch_size=64, target_update=10):
        self.state_size = state_size
        self.action_size = action_size
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        self.batch_size = batch_size

```

```

self.target_update = target_update
self.steps          = 0

self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
self.policy_net = DQNNetwork(state_size, action_size).to(self.device)
self.target_net = DQNNetwork(state_size, action_size).to(self.device)
self.target_net.load_state_dict(self.policy_net.state_dict())
self.target_net.eval()

self.optimizer = optim.Adam(self.policy_net.parameters(), lr=lr)
self.buffer     = ReplayBuffer()
self.loss_fn    = nn.SmoothL1Loss()

def select_action(self, state, valid_actions):
    """Epsilon-greedy voi valid action masking."""
    if random.random() < self.epsilon:
        return random.choice(valid_actions)
    state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
    with torch.no_grad():
        q_values = self.policy_net(state_t).squeeze(0).cpu().numpy()
    # Chi xet Q-values cua valid actions
    masked = np.full(self.action_size, -np.inf)
    for a in valid_actions:
        masked[a] = q_values[a]
    return int(np.argmax(masked))

def update(self):
    """Mot buoc gradient descent."""
    if len(self.buffer) < self.batch_size:
        return None
    s, a, r, ns, d = self.buffer.sample(self.batch_size)
    s = torch.FloatTensor(s).to(self.device)
    a = torch.LongTensor(a).unsqueeze(1).to(self.device)
    r = torch.FloatTensor(r).to(self.device)
    ns = torch.FloatTensor(ns).to(self.device)
    d = torch.FloatTensor(d).to(self.device)

    q_current = self.policy_net(s).gather(1, a).squeeze(1)
    with torch.no_grad():
        q_next = self.target_net(ns).max(1)[0]
    q_target = r + self.gamma * q_next * (1 - d)

    loss = self.loss_fn(q_current, q_target)
    self.optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(self.policy_net.parameters(), 1.0)
    self.optimizer.step()

```

```
# Cap nhat target network dinh ky
self.steps += 1
if self.steps % self.target_update == 0:
    self.target_net.load_state_dict(self.policy_net.state_dict())

self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)
return loss.item()
```

PPO Agent

python

```

class PPONetwork(nn.Module):
    """Actor-Critic network cho PPO."""
    def __init__(self, state_size, action_size, hidden=256):
        super().__init__()
        self.shared = nn.Sequential(
            nn.Linear(state_size, hidden), nn.Tanh(),
            nn.Linear(hidden, hidden),      nn.Tanh(),
        )
        self.actor = nn.Linear(hidden, action_size)
        self.critic = nn.Linear(hidden, 1)

    def forward(self, x):
        feat = self.shared(x)
        return self.actor(feat), self.critic(feat)

class PPOAgent:
    def __init__(self, state_size, action_size, lr=3e-4, gamma=0.99,
                  clip_eps=0.2, epochs=4, batch_size=64):
        self.gamma = gamma
        self.clip_eps = clip_eps
        self.epochs = epochs
        self.batch_size = batch_size
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.net = PPONetwork(state_size, action_size).to(self.device)
        self.optimizer = optim.Adam(self.net.parameters(), lr=lr)
        # Rollout buffers
        self.states, self.actions, self.rewards = [], [], []
        self.log_probs, self.values, self.dones = [], [], []

    def select_action(self, state, valid_actions):
        state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
        with torch.no_grad():
            logits, value = self.net(state_t)
        # Mask invalid actions bang -inf
        mask = torch.full((logits.shape[-1],), -1e9).to(self.device)
        for a in valid_actions:
            mask[a] = 0.0
        masked_logits = logits.squeeze(0) + mask
        dist = torch.distributions.Categorical(logits=masked_logits)
        action = dist.sample()
        self.states.append(state)
        self.actions.append(action.item())
        self.log_probs.append(dist.log_prob(action).item())
        self.values.append(value.item())
        return action.item()

```

```

def store_reward(self, reward, done):
    self.rewards.append(reward)
    self.dones.append(done)

def update(self):
    """PPO update tren rollout buffer."""
    if len(self.rewards) < self.batch_size:
        return None
    # Tinh returns (Monte Carlo)
    returns, G = [], 0.0
    for r, d in zip(reversed(self.rewards), reversed(self.dones)):
        G = r + self.gamma * G * (1 - d)
        returns.insert(0, G)

    returns = torch.FloatTensor(returns).to(self.device)
    advantages = returns - torch.FloatTensor(self.values).to(self.device)
    advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)

    states = torch.FloatTensor(np.array(self.states)).to(self.device)
    actions = torch.LongTensor(self.actions).to(self.device)
    old_lp = torch.FloatTensor(self.log_probs).to(self.device)

    losses = []
    for _ in range(self.epochs):
        logits, values = self.net(states)
        dist = torch.distributions.Categorical(logits=logits)
        new_lp = dist.log_prob(actions)
        ratio = (new_lp - old_lp).exp()
        surr1 = ratio * advantages
        surr2 = torch.clamp(ratio, 1 - self.clip_eps, 1 + self.clip_eps) * advantages
        actor_loss = -torch.min(surr1, surr2).mean()
        critic_loss = nn.MSELoss()(values.squeeze(-1), returns)
        entropy = dist.entropy().mean()
        loss = actor_loss + 0.5 * critic_loss - 0.01 * entropy
        self.optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.net.parameters(), 0.5)
        self.optimizer.step()
        losses.append(loss.item())

    # Clear buffers sau moi episode
    self.states, self.actions, self.rewards = [], [], []
    self.log_probs, self.values, self.dones = [], [], []
    return np.mean(losses)

```

Training Loop (200 servers, 5 VMs/server)

python

```

def collect_episode_metrics(env):
    """Tinh waiting time va completion time trung binh sau moi episode."""
    wait_times, comp_times = [], []
    for task in env.tasks:
        if task.status == "finished" and task.start_time is not None:
            wait_times.append(task.start_time - task.ready_time)
        if task.finish_time is not None and task.ready_time is not None:
            comp_times.append(task.finish_time - task.ready_time)
    return (np.mean(wait_times) if wait_times else 0.0,
            np.mean(comp_times) if comp_times else 0.0)

def train(agent, env, num_episodes=200, is_ppo=False, label="Agent"):
    reward_history, loss_history = [], []
    completion_hist, rejection_hist = [], []
    wait_hist = []

    for ep in range(num_episodes):
        state, info = env.reset(seed=ep)
        total_reward, losses = 0.0, []
        done = truncated = False

        while not (done or truncated):
            valid_actions = env.get_valid_actions()
            action = agent.select_action(state, valid_actions)
            next_state, reward, done, truncated, info = env.step(action)

            if is_ppo:
                agent.store_reward(reward, float(done or truncated))
            else:
                agent.buffer.push(state, action, reward, next_state,
                                   float(done or truncated))
                loss = agent.update()
                if loss is not None:
                    losses.append(loss)

            state = next_state
            total_reward += reward

        if is_ppo:
            loss = agent.update()
            if loss is not None:
                losses.append(loss)

    total = env.total_tasks
    rejected = info["rejected_tasks"]

```

```

reject_rate = rejected / total if total > 0 else 0.0
avg_wait, avg_comp = collect_episode_metrics(env)

reward_history.append(total_reward)
loss_history.append(np.mean(losses) if losses else 0.0)
rejection_hist.append(reject_rate)
completion_hist.append(avg_comp)
wait_hist.append(avg_wait)

```

```

if (ep + 1) % 20 == 0:
    print(f"[{label}] Episode {ep+1:4d} | Reward: {total_reward:8.2f} | "
          f"Completed: {info['completed_tasks']}/{total} | "
          f"Reject: {reject_rate*100:.1f}% | "
          f"AvgWait: {avg_wait:.2f} | AvgComp: {avg_comp:.2f}")

```

```

return reward_history, loss_history, completion_hist, rejection_hist, wait_hist

```

— Khoi tao moi truong —

```

env_dqn = CloudSchedulingEnv(
    workload_path="dataset.txt",
    num_servers=200,
    num_vms_per_server=5
)
env_ppo = CloudSchedulingEnv(
    workload_path="dataset.txt",
    num_servers=200,
    num_vms_per_server=5
)

```

— DQN —

```

dqn_agent = DQNAgent(
    state_size=env_dqn.state_size,
    action_size=env_dqn.action_space_size,
    lr=1e-3, gamma=0.99,
    epsilon=1.0, epsilon_min=0.05, epsilon_decay=0.995,
    batch_size=64, target_update=10
)
dqn_rewards, dqn_losses, dqn_comp, dqn_rej, dqn_wait = train(
    dqn_agent, env_dqn, num_episodes=200, is_ppo=False, label="DQN")

```

— PPO —

```

ppo_agent = PPOAgent(
    state_size=env_ppo.state_size,
    action_size=env_ppo.action_space_size,
    lr=3e-4, gamma=0.99,
    clip_eps=0.2, epochs=4, batch_size=64
)

```

```
)
ppo_rewards, ppo_losses, ppo_comp, ppo_rej, ppo_wait = train(
    ppo_agent, env_ppo, num_episodes=200, is_ppo=True, label="PPO")
```

Ve bieu do so sanh

```
python

def smooth(arr, w=10):
    """Moving average de lam muot duong cong."""
    return np.convolve(arr, np.ones(w) / w, mode='valid')

fig, axes = plt.subplots(2, 2, figsize=(14, 8))
fig.suptitle("DQN vs PPO - Training Metrics (200 servers, 5 VMs)", fontsize=14)

axes[0, 0].plot(smooth(dqn_rewards), label="DQN", color="steelblue")
axes[0, 0].plot(smooth(ppo_rewards), label="PPO", color="darkorange")
axes[0, 0].set_title("Reward Curve")
axes[0, 0].set_xlabel("Episode"); axes[0, 0].legend(); axes[0, 0].grid(True)

axes[0, 1].plot(smooth(dqn_losses), label="DQN", color="steelblue")
axes[0, 1].plot(smooth(ppo_losses), label="PPO", color="darkorange")
axes[0, 1].set_title("Loss Curve")
axes[0, 1].set_xlabel("Episode"); axes[0, 1].legend(); axes[0, 1].grid(True)

axes[1, 0].plot(smooth(dqn_rej), label="DQN", color="steelblue")
axes[1, 0].plot(smooth(ppo_rej), label="PPO", color="darkorange")
axes[1, 0].set_title("Rejection Rate")
axes[1, 0].set_xlabel("Episode"); axes[1, 0].legend(); axes[1, 0].grid(True)

axes[1, 1].plot(smooth(dqn_wait), label="DQN Wait", color="steelblue", linestyle='--')
axes[1, 1].plot(smooth(ppo_wait), label="PPO Wait", color="darkorange", linestyle='--')
axes[1, 1].plot(smooth(dqn_comp), label="DQN Comp", color="steelblue", linestyle='--')
axes[1, 1].plot(smooth(ppo_comp), label="PPO Comp", color="darkorange", linestyle='--')
axes[1, 1].set_title("Avg Waiting & Completion Time")
axes[1, 1].set_xlabel("Episode"); axes[1, 1].legend(); axes[1, 1].grid(True)

plt.tight_layout()
plt.savefig("training_curves.png", dpi=150)
plt.show()
```

Cau 4: Evaluation tren testset.txt

python

```

def evaluate(agent, workload_path, num_servers=200, num_vms=5,
            is_ppo=False, label="Agent"):
    """Danh gia policy tren test set – khong co exploration."""
    env_test = CloudSchedulingEnv(
        workload_path=workload_path,
        num_servers=num_servers,
        num_vms_per_server=num_vms
    )
    state, _ = env_test.reset(seed=0)

    # Tat exploration
    if not is_ppo:
        agent.epsilon = 0.0

    done = truncated = False
    total_reward = 0.0

    while not (done or truncated):
        valid_actions = env_test.get_valid_actions()
        with torch.no_grad():
            action = agent.select_action(state, valid_actions)
        state, reward, done, truncated, info = env_test.step(action)
        total_reward += reward

    # Thu thap metrics
    completed = info["completed_tasks"]
    rejected = info["rejected_tasks"]
    total = env_test.total_tasks
    reject_rate = rejected / total if total > 0 else 0.0
    avg_wait, avg_comp = collect_episode_metrics(env_test)

    print(f"\n=== {label} – Testset Evaluation ===")
    print(f"  Total tasks      : {total}")
    print(f"  Completed         : {completed}  ({100*(1-reject_rate):.1f}%)")
    print(f"  Rejected          : {rejected}    ({reject_rate*100:.1f}%)")
    print(f"  Avg Wait Time     : {avg_wait:.4f}")
    print(f"  Avg Comp Time      : {avg_comp:.4f}")
    print(f"  Total Reward      : {total_reward:.2f}")

    return dict(completed=completed, rejected=rejected,
                reject_rate=reject_rate,
                avg_wait=avg_wait, avg_comp=avg_comp,
                total_reward=total_reward)

```

```

# Chay evaluation

```

```
dqn_metrics = evaluate(dqn_agent, "testset.txt", is_ppo=False, label="DQN")
ppo_metrics = evaluate(ppo_agent, "testset.txt", is_ppo=True, label="PPO")

# In bang tong hop
print("\n{'='*55}")
print(f"{'Metric':<25} {'DQN':>12} {'PPO':>12}")
print(f"{'='*55}")
for key in ["reject_rate", "avg_wait", "avg_comp", "total_reward"]:
    print(f"{key:<25} {dqn_metrics[key]:>12.4f} {ppo_metrics[key]:>12.4f}")
print(f"{'='*55}")
```

Cau 7: Phan Tich va Ly Giai Ket Qua

Diem manh — DQN

- **Sample efficiency cao:** Replay buffer cho phep tai su dung moi experience nhieu lan, phu hop khi so episode bi han che.
- **On dinh:** Target network giu gradient khong bi oscillation, tranh diverge khi reward co the sparse.
- **Valid action masking:** Ket hop tot voi epsilon-greedy — chi can mask Q-values cua action khong hop le, agent hoc tap trung vao khong gian action thuc su co y nghia.
- **Rejection rate thap hon:** Do sample efficiency cao, DQN dat rejection rate thap hon PPO sau cung so episodes, giam so task bi bo.

Diem yeu — DQN

- **Hoi tu cham hon:** Can nhieu episode warm-up de fill replay buffer truoc khi gradient bat dau co ich.
- **Nhat cam voi hyperparameter:** Hoc tot hay khong phu thuoc nhieu vao learning rate, batch size, epsilon decay.

Diem manh — PPO

- **Hoi tu nhanh:** On-policy, reward signal duoc su dung ngay lap tuc trong cung episode, cac episode dau hoi tu tot hon DQN.
- **Entropy bonus:** Tu dong khuyen khich exploration ma khong can tuning epsilon riêng.
- **On dinh qua tung update:** Clipped surrogate objective gioi han buoc update, tranh policy collapse.

Diem yeu — PPO

- **Sample efficiency thap hon:** Moi rollout chi duoc dung mot so epochs nho roi bo, lang phi experience.

- **Rejection rate cao hơn DQN** trong cùng số episodes do học chậm hơn trên không gian action lớn (1001 actions).

Kết luận tổng hợp

Metric	DQN	PPO	Mục tiêu
Rejection Rate	Thấp hơn	Trung bình	Càng nhỏ càng tốt
Avg Waiting Time	Thấp	Thấp	Càng nhỏ càng tốt
Avg Completion Time	Thấp	Trung bình	Càng nhỏ càng tốt
Sample Efficiency	Cao	Trung bình	Càng cao càng tốt
Tốc độ hội tụ ban đầu	Chậm	Nhanh	Càng nhanh càng tốt

DQN là lựa chọn tốt hơn cho bài toán này do:

1. Action space rời rạc lớn (1001 actions) — `argmax` trên Q-values hiệu quả hơn Categorical sampling.
2. Replay buffer giúp học tốt khi số episode bị giới hạn bởi kích thước dataset.
3. Đạt rejection rate thấp hơn và completion time ngắn hơn sau cùng số episodes.

PPO vẫn có giá trị làm baseline và so sánh, đặc biệt khi cần kết quả nhanh ở các episode đầu.

Ghi chú: Hệ thống 200 servers, 5 VMs/server. Training trên `dataset.txt` (427,631 tasks).
Evaluation trên `testset.txt` (99,999 tasks).